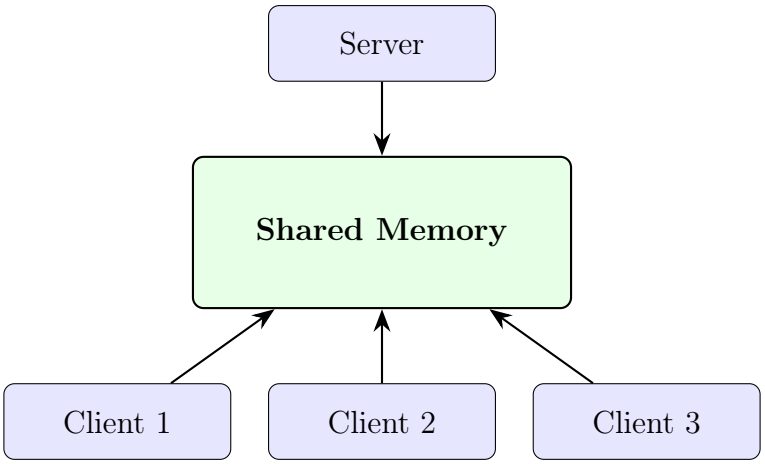
Shared Memory Chat System

An Inter-Process Communication Project



Course:	Operating Systems
Language:	C (POSIX)
Platform:	Linux
Date:	April 13, 2026

Contents

1 Introduction **2**
1.1 Project Objectives . . . 2
1.2 System Requirements . . . 2

2 Architecture Overview **2**
2.1 System Components . . . 2
2.2 Architecture Diagram . . . 3
2.3 File Structure . . . 3

3 OS Concepts Used **3**
3.1 Shared Memory . . . 3
3.2 Semaphores . . . 4
3.3 Process Creation with fork() . . . 5
3.4 Signals . . . 5
3.5 Circular Buffer . . . 5

4 Data Structures **6**
4.1 Memory Layout . . . 6
4.2 Size Calculation . . . 6

5 Detailed Code Walkthrough **6**
5.1 Server: Startup Sequence . . . 6
5.2 Client: Connection Sequence . . . 7
5.3 Message Sending (Critical Section) . . . 8
5.4 Message Receiving (Timed Wait) . . . 8
5.5 Server: Shutdown Sequence . . . 9
5.6 Client: Disconnect Sequence . . . 9

6 Complete Data Flow **10**
6.1 Message Flow Diagram . . . 10

7 Building and Running **10**
7.1 Build Commands . . . 10
7.2 Running the System . . . 10
7.3 Cleanup After Crash . . . 11

8 Summary of OS Concepts **11**

9 Possible Enhancements **11**

Introduction

This document describes the design and implementation of a **Shared Memory Chat System** built in C. The system demonstrates core operating systems concepts including inter-process communication (IPC), process synchronization, shared memory management, and semaphore-based mutual exclusion.

Project Objectives

- Implement inter-process communication using POSIX shared memory
- Demonstrate mutual exclusion using named semaphores
- Use process creation (`fork()`) for concurrent read/write operations
- Implement a circular buffer for efficient message storage
- Handle process signals for graceful shutdown

System Requirements

- Linux operating system (POSIX compliant)
- GCC compiler
- POSIX shared memory support (`sys/shm.h`)
- POSIX semaphore support (`semaphore.h`)

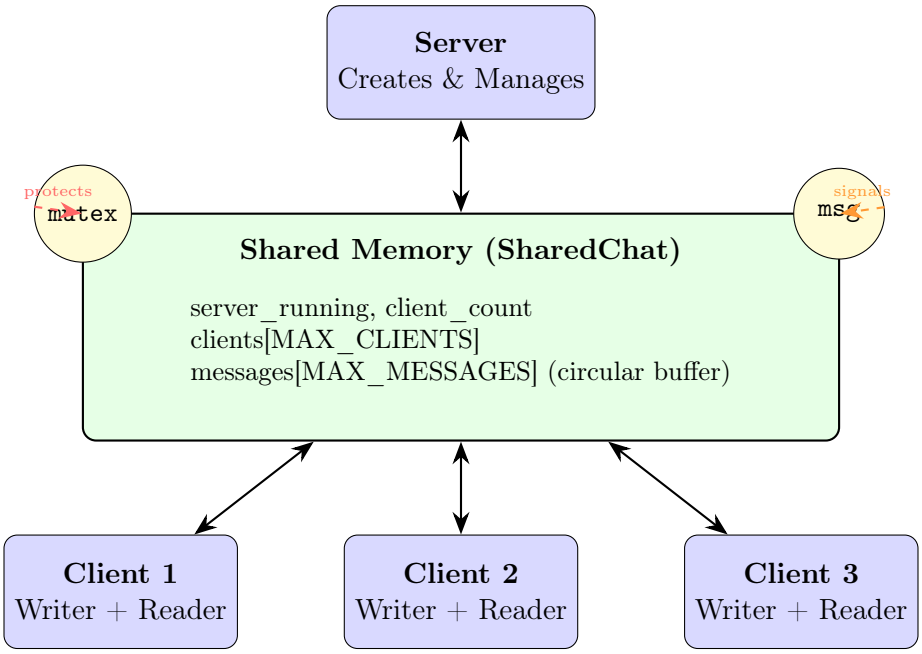
Architecture Overview

System Components

The system consists of three main components:

1. **Server Process:** Creates and manages shared memory and semaphores. Provides administrative commands to monitor the chat system.
2. **Client Processes:** Connect to the shared memory, register themselves, and exchange messages through the shared buffer.
3. **Shared Memory Segment:** A kernel-managed memory block accessible by all processes, containing client information and a circular message buffer.

Architecture Diagram



File Structure

File	Purpose	Lines
`common.h`	Shared data structures and constants	∼ $50
`server.c`	Chat server — creates & manages IPC resources	∼ $160
`client.c`	Chat client — connects, sends, receives messages	∼ $200
`Makefile`	Build system	∼ $20

OS Concepts Used

Shared Memory

Shared Memory (System V IPC)

Shared memory allows multiple processes to access the same region of physical memory. It is the **fastest form of IPC** because data does not need to be copied between processes — all processes read from and write to the same memory location.

Key functions used:

- `shmget(key, size, flags)` — Create or find a shared memory segment
- `shmat(id, NULL, 0)` — Attach the segment to process address space
- `shmdt(ptr)` — Detach from shared memory
- `shmctl(id, IPC_RMID, NULL)` — Remove the segment from the system

How it works in our system:

1. The server calls `shmget()` with `IPC_CREAT` to create a new shared memory segment identified by key `0x1234`.
2. The server attaches to this memory using `shmat()` and initializes it.
3. Clients call `shmget()` with the same key (without `IPC_CREAT`) to find the existing segment.
4. Clients attach using `shmat()` — now both server and clients have pointers to the **same physical memory**.
5. Any write by one process is **instantly visible** to all other attached processes.

Semaphores

POSIX Named Semaphores

A semaphore is a synchronization primitive that controls access to shared resources. It maintains an integer value that can be atomically incremented (`sem_post`) or decremented (`sem_wait`). If a process tries to decrement a semaphore whose value is 0, it **blocks** until another process increments it.

Two semaphores are used:

Name	Initial Value	Purpose	Type
`mutex_sem`	1	Mutual exclusion for shared data	Binary semaphore
`msg_sem`	0	Notification of new messages	Counting semaphore

Mutex Semaphore — Preventing Race Conditions:

Without synchronization, two processes modifying shared data simultaneously could corrupt it. Consider two clients registering at the same time:

```
Without mutex:                      With mutex:
  Client A reads count=0              Client A: sem_wait() LOCK
  Client B reads count=0              Client A reads count=0
  Client A writes slot 0              Client A writes slot 0
  Client B writes slot 0 (!)          Client A: count=1
  Client A: count=1                   Client A: sem_post() UNLOCK
  Client B: count=1 (should be 2!)    Client B: sem_wait() LOCK
                                      Client B reads count=1
  RESULT: Data corrupted!             Client B writes slot 1
                                      Client B: count=2
                                      Client B: sem_post() UNLOCK
                                      RESULT: Correct!
```

Message Semaphore — Efficient Notification:

Instead of readers continuously polling shared memory (busy waiting), the message semaphore allows readers to **sleep** until a writer signals that a new message is available.

```
Writer (Alice):                   Reader (Bob):
  Writes message to buffer          sem_timedwait(msg_sem)
  sem_post(msg_sem)     Wakes up!
  (value: 0 → 1)                    Reads new message
                                    (value: 1 → 0)
```

Process Creation with fork()

fork() System Call

`fork()` creates a new process by duplicating the calling process. The new process (child) is an exact copy of the parent, including all variables, open file descriptors, and attached shared memory segments. After `fork()`, parent and child execute **concurrently**.

Why fork() is needed:
Each client needs to simultaneously:

1. **Wait for user input** (`fgets()` blocks until Enter is pressed)
2. **Watch for new messages** (must check shared memory continuously)

A single process cannot do both because `fgets()` blocks. The solution is to `fork()` into two processes:

- **Parent process** → Writer: reads user input and writes to shared memory
- **Child process** → Reader: monitors shared memory and displays new messages

Both processes share the same attached shared memory and semaphores.

Signals

UNIX Signals

Signals are software interrupts sent to a process to notify it of events. A process can register **signal handlers** — functions that execute when a specific signal is received, interrupting the normal flow of execution.

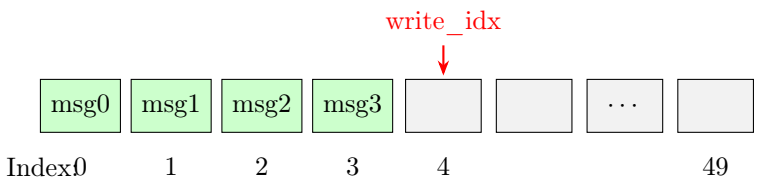
Signals used:

Signal	Triggered By	Purpose
`SIGINT`	User presses Ctrl+C	Graceful cleanup and exit
`SIGTERM`	`kill` command	Terminate reader child process
`SIGUSR1`	Server shutdown	Notify clients that server is closing

Circular Buffer

Circular Buffer

A circular buffer (ring buffer) is a fixed-size data structure that wraps around. When the write index reaches the end, it wraps to the beginning, overwriting the oldest data. This provides $O(1)$ write operations without needing to shift elements.



The formula `write_idx = (write_idx + 1) % MAX_MESSAGES` ensures automatic wrapping:

$$\text{idx} = 48 \Rightarrow (48 + 1) \mod 50 = 49$$
$$\text{idx} = 49 \Rightarrow (49 + 1) \mod 50 = 0 \quad \text{(wraps to beginning)}$$

Data Structures

Memory Layout

The `SharedChat` structure defines the layout of shared memory:

```
typedef struct {
    int          server_running;
    int          client_count;
    int          message_count;
    int          msg_write_idx;
    ClientInfo   clients[MAX_CLIENTS];
    Message      messages[MAX_MESSAGES];
} SharedChat;
```

Listing 1: SharedChat Structure

Size Calculation

Field	Size (bytes)
`server_running` (int)	4
`client_count` (int)	4
`message_count` (int)	4
`msg_write_idx` (int)	4
`clients[10]` (10 × ClientInfo)	≈ 400
`messages[50]` (50 × Message)	≈ 15,000
Total	**≈ 15,416 bytes**

Detailed Code Walkthrough

Server: Startup Sequence

1. **Register signal handlers:** `signal(SIGINT, cleanup)` ensures Ctrl+C triggers graceful shutdown instead of abrupt termination.

2. **Create shared memory:**

```
shm_id = shmget(SHM_KEY, sizeof(SharedChat), IPC_CREAT | 0666)
    ;
```

   The kernel allocates a memory block of size `sizeof(SharedChat)` bytes, accessible by any process that knows the key `0x1234`. The flag `IPC_CREAT` creates the segment if it doesn't exist. Permission `0666` allows read/write for all users.

3. **Attach shared memory:**

```
shared = (SharedChat *)shmat(shm_id, NULL, 0);
```

Maps the shared segment into the server's virtual address space. Returns a pointer that can be used like any regular pointer.

4. **Initialize shared memory:**

```
memset(shared, 0, sizeof(SharedChat));
shared->server_running = 1;
```

Zeros out all fields, then sets the server status flag. All client slots and message slots start as inactive.

5. **Create semaphores:**

```
mutex_sem = sem_open(SEM_MUTEX_NAME, O_CREAT, 0666, 1);
msg_sem   = sem_open(SEM_MSG_NAME,   O_CREAT, 0666, 0);
```

Named semaphores are created in the filesystem (`/dev/shm/`). The mutex starts at 1 (unlocked), the message semaphore starts at 0 (no messages).

6. **Enter command loop:** Server waits for administrative commands (`list`, `msgs`, `check`, `quit`).

Client: Connection Sequence

1. **Get username:** Read from standard input using `fgets()`.

2. **Find shared memory:**

```
shm_id = shmget(SHM_KEY, sizeof(SharedChat), 0666);
```

Note: **No `IPC_CREAT` flag**. The client expects the server to have already created the segment. If the segment doesn't exist, `shmget()` returns -1.

3. **Attach and verify:** Attach to shared memory, then check `shared->server_running == 1`.

4. **Open existing semaphores:**

```
mutex_sem = sem_open(SEM_MUTEX_NAME, 0);
```

Flag `0` means "open existing" — do not create.

5. **Register in client array:**

```
sem_wait(mutex_sem);          // Lock
// Find empty slot, fill in PID and name
shared->clients[i].active = 1;
shared->client_count++;
sem_post(mutex_sem);          // Unlock
```

The entire registration is wrapped in a critical section to prevent race conditions.

6. **Fork into reader and writer:**

```
pid_t pid = fork();
if (pid == 0)     reader_process();    // Child
else              writer_loop();        // Parent
```

Message Sending (Critical Section)

```
void send_message(const char *name, const char *content) {
    sem_wait(mutex_sem);     // Enter critical section

    int idx = shared->msg_write_idx;
    shared->messages[idx].active = 1;
    shared->messages[idx].sender_pid = getpid();
    strncpy(shared->messages[idx].sender_name, name, MAX_NAME_LEN
        - 1);
    strncpy(shared->messages[idx].content, content, MAX_MSG_LEN -
        1);
    shared->messages[idx].timestamp = time(NULL);
    shared->msg_write_idx = (idx + 1) % MAX_MESSAGES;
    shared->message_count++;

    sem_post(mutex_sem);     // Leave critical section
    sem_post(msg_sem);       // Signal: new message available
}
```

Listing 2: send_message() function

Critical Section Analysis

The code between `sem_wait(mutex_sem)` and `sem_post(mutex_sem)` is a **critical section**. Only one process can execute this code at a time. This prevents:

- Two writers using the same `msg_write_idx`
- A reader seeing a half-written message
- The write index being corrupted by concurrent increments

Message Receiving (Timed Wait)

```
while (running && shared->server_running) {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += 1;     // 1-second timeout

    if (sem_timedwait(msg_sem, &ts) == 0) {
        sem_wait(mutex_sem);     // Lock
        // Read and display message
        sem_post(mutex_sem);     // Unlock
    }
    // Timeout: loop back, check running flags
```

```
}
```

Listing 3: Reader process loop

`sem_timedwait()` is used instead of `sem_wait()` because:

- `sem_wait()` blocks **indefinitely** — the reader could never check if the server shut down.
- `sem_timedwait()` returns after 1 second even if no message arrives, allowing the loop to check the `running` and `server_running` flags.

Server: Shutdown Sequence

1. Set `shared->server_running = 0` — clients will see this on next loop iteration.
2. Send `SIGUSR1` to all active client PIDs — wakes up blocked clients.
3. Detach from shared memory (`shmdt`).
4. Remove shared memory from kernel (`shmctl` with `IPC_RMID`).
5. Close and unlink (delete) both semaphores.

Client: Disconnect Sequence

1. Mark own slot as `active = 0` in shared memory (inside critical section).
2. Decrement `client_count`.
3. Parent sends `SIGTERM` to child (reader process).
4. Parent calls `waitpid()` to reap child and prevent zombie process.
5. Detach from shared memory (`shmdt`).
6. Close (but do **not** unlink) semaphores — only server unlinks.

Complete Data Flow

Message Flow Diagram

1. Alice types "Hello!"
2. fgets() returns input
3. send_message() called
4. sem_wait(mutex) — LOCK *Critical section begin*
5. Write to messages[idx]
6. sem_post(mutex) — UNLOCK *Critical section end*
7. sem_post(msg_sem) — NOTIFY *Notification signal*
8. Bob's reader wakes up
9. sem_wait(mutex) — LOCK
10. Read message, display it
11. sem_post(mutex) — UNLOCK

Building and Running

Build Commands

```
# Build both server and client
make

# Or build individually
make server
make client
```

Listing 4: Building the project

Running the System

```
# Terminal 1: Start the server
./server
```

```

# Terminal 2: Start first client
./client
# Enter username: Alice

# Terminal 3: Start second client
./client
# Enter username: Bob
```

Listing 5: Running the chat system

Cleanup After Crash

If the server crashes without proper cleanup, IPC resources remain in the kernel:

```
# Remove leftover IPC resources
make purge

# Or manually:
ipcrm -M 0x1234                          # Remove shared memory
rm -f /dev/shm/sem.chat_mutex            # Remove mutex semaphore
rm -f /dev/shm/sem.chat_msg              # Remove message semaphore
```

Listing 6: Manual cleanup

Summary of OS Concepts

Concept	Implementation	Purpose
Shared Memory	`shmget/shmat/shmdt/shmctl`	IPC between all processes
Semaphores	`sem_open/sem_wait/sem_post`	Mutual exclusion & signaling
Processes	`fork()`	Concurrent reader + writer
Signals	`signal/kill`	Shutdown notification
Circular Buffer	Modulo arithmetic	Efficient fixed-size storage
Critical Sections	Semaphore lock/unlock	Prevent race conditions
Zombie Prevention	`waitpid()`	Clean process termination

Possible Enhancements

1. **Private Messaging:** Add a `recipient_pid` field to the Message struct.
2. **Multiple Chat Rooms:** Use separate shared memory segments per room.
3. **Threads Instead of Processes:** Replace `fork()` with `pthread_create()`.
4. **Message Persistence:** Log messages to a file for history.
5. **User Authentication:** Add password verification before joining.